THIRUVENKADAPRASATH.S

192312016

15	Implement Iris Flower Classification using Naive Bayes classifier

CSV FILE

SepalLength,SepalWidth,PetalLength,PetalWidth,Species

5.1,3.5,1.4,0.2,Setosa

4.9,3.0,1.4,0.2,Setosa

4.7,3.2,1.3,0.2,Setosa

7.0,3.2,4.7,1.4,Versicolor

6.4,3.2,4.5,1.5,Versicolor

6.9,3.1,4.9,1.5,Versicolor

6.3,3.3,6.0,2.5,Virginica

5.8,2.7,5.1,1.9,Virginica

7.1,3.0,5.9,2.1,Virginica

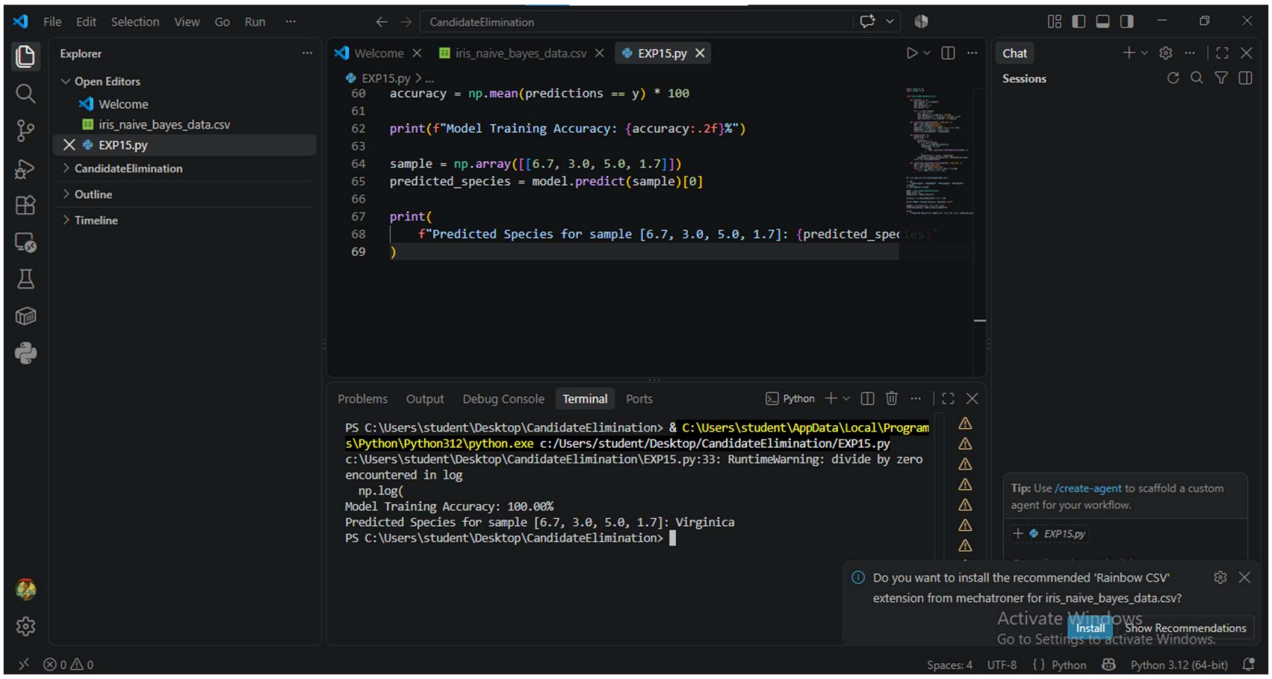
PROGRAM

```
import numpy as np
import pandas as pd



class GaussianNaiveBayesScratch:

    def fit(self, X, y):
        self.classes = np.unique(y)
        self.means = {}
        self.variances = {}
        self.priors = {}
```

```
    for c in self.classes:
        X_c = X[y == c]
        self.means[c] = np.mean(X_c, axis=0)
        self.variances[c] = np.var(X_c, axis=0) + 1e-9
        self.priors[c] = X_c.shape[0] / X.shape[0]

def _calculate_likelihood(self, class_val, x):
    mean = self.means[class_val]
    var = self.variances[class_val]
    numerator = np.exp(-((x - mean) ** 2) / (2 * var))
    denominator = np.sqrt(2 * np.pi * var)
    return np.prod(numerator / denominator)

def predict(self, X):
    predictions = []
    for x in X:
        posteriors = {}
        for c in self.classes:
            prior = np.log(self.priors[c])
            likelihood = np.sum(
                np.log(
                    self._calculate_likelihood_vectorized(c, x)
                )
            )
            posteriors[c] = prior + likelihood
        predictions.append(max(posteriors, key=posteriors.get))
    return np.array(predictions)

def _calculate_likelihood_vectorized(self, class_val, x):
```

```
        mean = self.means[class_val]
        var = self.variances[class_val]
        return (1 / np.sqrt(2 * np.pi * var)) * np.exp(
            -((x - mean) ** 2) / (2 * var)
        )


df = pd.read_csv("iris_naive_bayes_data.csv")

X = df[
    ["SepalLength", "SepalWidth", "PetalLength", "PetalWidth"]
].values
y = df["Species"].values

model = GaussianNaiveBayesScratch()
model.fit(X, y)
predictions = model.predict(X)

accuracy = np.mean(predictions == y) * 100

print(f"Model Training Accuracy: {accuracy:.2f}%")

sample = np.array([[6.7, 3.0, 5.0, 1.7]])
predicted_species = model.predict(sample)[0]

print(
    f"Predicted Species for sample [6.7, 3.0, 5.0, 1.7]: {predicted_species}"
)
```

OUTPUT


File Edit Selection View Go Run
CandidateElimination
Explorer
Open Editors
Welcome
iris_naive_bayes_data.csv
EXP15.py
CandidateElimination
Outline
Timeline
Welcome
iris_naive_bayes_data.csv
EXP15.py
EXP15.py > ...
60 accuracy = np.mean(predictions == y) * 100
61
62 print(f"Model Training Accuracy: {accuracy:.2f}%")
63
64 sample = np.array([[6.7, 3.0, 5.0, 1.7]])
65 predicted_species = model.predict(sample)[0]
66
67 print(
68 f"Predicted Species for sample [6.7, 3.0, 5.0, 1.7]: {predicted_spec
69)
Problems Output Debug Console Terminal Ports
Python
PS C:\Users\student\Desktop\CandidateElimination> & C:\Users\student\AppData\Local\Programs\Python\Python312\python.exe c:/Users/student/Desktop/CandidateElimination/EXP15.py
c:\Users\student\Desktop\CandidateElimination\EXP15.py:33: RuntimeWarning: divide by zero encountered in log
np.log(
Model Training Accuracy: 100.00%
Predicted Species for sample [6.7, 3.0, 5.0, 1.7]: Virginica
PS C:\Users\student\Desktop\CandidateElimination>
Chat
Sessions
Tip: Use /create-agent to scaffold a custom agent for your workflow.
EXP15.py
Do you want to install the recommended 'Rainbow CSV' extension from mechatroner for iris_naive_bayes_data.csv?
Install
Show Recommendations
Activate Windows
Go to Settings to activate Windows.
Spaces: 4 UTF-8 Python Python 3.12 (64-bit)